Big-O: Why We Count Operations, Not Seconds

Programming and Data Structures — Week 1, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain why algorithm cost is measured in operations rather than seconds, classify a piece of code into one of the five common complexity classes covered today, and predict, before ever running it, which of two competing approaches will slow down first as the input grows.

Outline

Today covers: the contact list problem that motivates the whole lecture, five complexity classes each anchored to a real example, a chart that makes the difference between them visual rather than abstract, an in-class quiz where you classify five code snippets on the spot, and a closing summary.

The problem: finding a contact

Your phone has 1,000 contacts, stored in whatever order you added them — not alphabetical, not by anything useful. You type "Amina" into search. How does the phone find her? There are two candidate approaches, and the rest of this lecture is about how to tell, in advance, which one to use.

```
contacts = ["Zara", "Amina", "John", "Priya", ...]  # 1,000 names, unsorted

def find_linear(contacts, name):
    for i, c in enumerate(contacts):
        if c == name:
            return i
    return -1
```

Class 1 — O(1): the speed-dial slot

```
speed_dial = {1: "Mum", 2: "Amina", 3: "Work"}
print(speed_dial[2])   # "Amina" - one lookup, regardless of dict size
```

O(1), constant time. Grabbing item 2 out of a Python dict costs the same whether the dict holds 10 entries or 10 million — the cost does not depend on the size of the input at all. Array indexing (`arr[5]`) and dictionary lookup are the two examples you will use constantly in this course.

Class 2 — O(log n): the guessing game

Play this in class: pick a number from 1 to 1,000. A student guesses; you say only "higher" or "lower." How many guesses does it take in the worst case? The answer is about 10 — not 1,000, not 500. Each guess eliminates half of what remains, so the count needed is $\log_2(1000) \approx 10$.

```
def binary_search(sorted_contacts, name):
    lo, hi = 0, len(sorted_contacts) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if sorted_contacts[mid] == name:
            return mid
        elif sorted_contacts[mid] < name:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

This is exactly the guessing-game strategy, applied to a **sorted** contact list. It only works because the list is sorted — you cannot ask "higher or lower" of a shuffled deck.

Class 3 — O(n): checking every contact

```
def find_linear(contacts, name):
    for i, c in enumerate(contacts):
        if c == name:
            return i
    return -1
```

This is our opening example. In the worst case — the name is last in the list, or not present at all — this checks every one of the n contacts. Double the contact list, and in the worst case you double the number of checks. Cost grows in a straight line with input size, which is what "linear" means.

Class 4 — O(n log n): sorting the list first

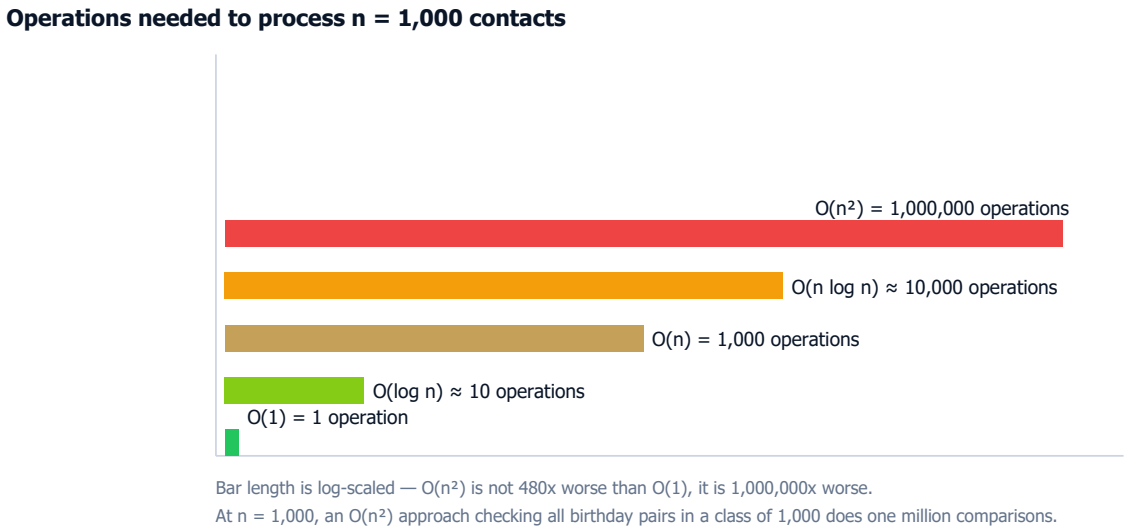
Binary search (Class 2) has a hidden requirement: the list must already be sorted. Sorting 1,000 unsorted contacts costs O(n log n) using a good algorithm — merge sort, which you meet properly in Week 2 — more expensive than a single linear scan, but far cheaper than the naive alternative below. It pays for itself the moment you search the same list more than once.

Class 5 — O(n²): comparing every pair

A different, common problem: does any pair of students in a class of 1,000 share a birthday? The naive approach compares every student to every other student — for n students, that is roughly $n^2/2$ comparisons. At n = 1,000, that is close to 500,000 comparisons for a question a hash set answers in a single O(n) pass. O(n²) is what nested loops over the same data usually cost, and it is the first complexity class where "just use a faster machine" stops being a real fix.

```
def has_duplicate_birthday(birthdays):
    n = len(birthdays)
    for i in range(n):
        for j in range(i + 1, n):
            if birthdays[i] == birthdays[j]:
                return True
    return False
```

The chart that makes it visual



Bar length is log-scaled — O(n²) is not 480x worse than O(1), it is 1,000,000x worse.
At n = 1,000, an O(n²) approach checking all birthday pairs in a class of 1,000 does one million comparisons.

At n = 1,000, O(1) costs one operation and O(n²) costs one million — a factor of a million, not a small difference you can shrug off with better hardware.

Quiz — classify these, right now

Before reading on, classify each of these three functions yourself: `a(items)` returns `items[0]`; `b(items, target)` loops once through `items` looking for `target`; `c(items)` has a loop nested inside a loop, both over `items`. Write down O(1), O(n), or O(n²) for each before checking the answer on the next slide.

Quiz — answers

`a` is O(1): a single index access that costs the same no matter how long `items` is. `b` is O(n): a linear search whose worst case scans every element once. `c` is O(n²): a loop nested inside another loop, both running over the same n items, giving roughly $n \times n$ operations.

Summary

We count operations rather than seconds because seconds are a fact about the machine, not the algorithm — the same code takes a different number of seconds on a laptop, a phone, or after a software update, but the number of operations it performs for a given input size does not change. In order of how badly they scale as input grows: O(1), then O(log n), then O(n), then O(n log n), then O(n²).

Choosing a data structure well, throughout the rest of this course, is really choosing which of these growth rates you are willing to live with. Next week: arrays and dynamic arrays, the first structure in this course we will measure this formally.